About this PDF

Built only from: **NOC:Cloud Computing and Distributed Systems (NPTEL course book, week 4)** — https://nptel.ac.in/courses/106104182

Licence: CC BY-SA 4.0. Accessed 2026-09-26.

Covers 12 of 12 syllabus sub-topics of this unit.

Partly covered in the source:

- Message Ordering and Group Communication: Message Ordering Paradigms

Logical Time: Physical Clock Synchronization: NTP

TimeandClockSynchronizationinclouddatacenters. (ReferSlideTime:00:19) Preface; content of this lecture. We will discuss the fundamentals of clock synchronization, in cloud and also it is different algorithms. Before that we will discuss the importance of time in the distributed system and the cloud systems. We will also cover causality a general framework to avoid the physical clock synchronizations using logical clocks. And we will cover the different logical clock such as Lamport clock, vector clocks to capture the causality between the events in the distributed system and thecloudsystem. (ReferSlideTime:01:21) So, letusseetheimportanceoftime.So,timeisoneofthemostcrucialinthedistributed system, and as also in the cloud system. But the sup

A Framework for a System of Logical Clocks Scalar Time

Definition

Scalar time is a logical clock that assigns a single number to each event in a distributed system.

Explanation

1. Within the same process, if event a occurs before event b, then the local counter of a is less than that of b.
2. If process p sends a message m to process q, then the send event of m happens before the receive event of m.
3. Transitivity: if event a happens before b and b happens before c, then a happens before c. These rules create a partial order among all events, allowing requests for a critical section to be executed in increasing order of their scalar timestamps.

Example

- P1 has events A, B, C, D, E; event B is a send of a message.
- P2 receives the message from P1 at event F.
- P3 receives a message from P1 at event J. These events illustrate how the scalar timestamps and the happened-before rules order events across processes.

Algorithm / Steps

1. Each process maintains a local counter that is incremented before each event.

2. When sending a message, the process attaches its current counter value as the timestamp.
3. Upon receiving a message, the receiving process updates its counter to be greater than both its own counter and the received timestamp.
4. Requests for the critical section are served in increasing order of the scalar timestamps.

Key points

- Scalar time provides a single numeric value for each event.
- The happened-before relation is defined by three rules: local order, send-before-receive, and transitivity.
- These rules create a partial order that can be used to schedule critical-section requests.
- Each process updates its counter locally and with message exchanges, ensuring consistent ordering across the system.

Vector Time

LetusseethroughtheexampleCandFalthoughtheyaretimestampis3.So,doweinfer that C has happened beforeF?Similarly,HandCHandC;so,thetimeofHis1andthe time of C is 3. So, can be infer that H has happened before C. No, we cannot, why because they are not causally related. Hence they do not follow this happened before relation. Hence they are called as concurrent events. So, C F and H C they arethepairs which are not obeying the causality relation. Hence they are called concurrent events hereinthiscase. (ReferSlideTime:70:55) So, a pair of concurrent even does not have the causal path from one event to another. So, lamport timestamp not guaranteed to be ordered or unequal for concurrent events. This is sincetheconcurrenteventsarenotcausallyrelated.Bu

Message Ordering and Group Communication: Message Ordering Paradigms

Partly covered in the source.

Now, here every site S i keeps a queue and this is called a request queue i which contains the mutual exclusion request which are ordered by their timestamps, this algorithm requires the communication channel to deliver the messages in FIFO order, there are 3 different type of messages which are used in this algorithm, they are request reply and release. These messages with timestamp also updates the logical clock. Let us see the algorithm called Lamport's algorithm, the first phase first step is called requesting the critical section. So, when a site S i want to enter in a critical section, it broadcasts a request message with a timestamp and the id of a process i. This particular message is broadcasts to all other sites and also it places

Asynchronous Execution with Synchronous Communication

Definition

The asynchronous distributed system model consists of processes that communicate over a network without a shared global clock. The state of each process includes data, stack, heap, program counter, and registers. The state changes via internal events (instruction execution), sending a message, or receiving a message. Each process has a local clock that timestamps internal events.

Explanation

In this model, messages are the only means of communication. Delays of messages and processing are unpredictable and unbounded, unlike in a synchronous model where a shared memory and bounded message delays exist. Clock synchronization across hosts is nontrivial.

Key points

- Processes communicate only via messages.
- Message delays and processing delays are unbounded.
- Each process has a local clock for timestamping internal events.
- No shared global clock exists in the system.

Synchronous Program Order on Asynchronous System

Definition

Synchronous program order on an asynchronous system is the logical ordering of events that imposes a partial order on all events in the system using Lamport's happened-before relation, which is defined by three rules.

Explanation

Because processes in an asynchronous distributed system do not share a global clock and message delays are unbounded, physical clocks cannot provide a reliable ordering of events. Logical clocks, such as Lamport timestamps, are therefore used to assign a scalar value to each event so that the causal relationship between events can be inferred. The happened-before relation is established by: (1) ordering events on the same process by their local logical clock values; (2) ordering a send event before the corresponding receive event of the same message; and (3) transitivity of the relation. These rules create a partial order among all events, capturing causality without requiring synchronized physical time.

Example

Consider a person who unlocks a door before entering the house. The unlocking event must occur before the entering event, so the unlocking event happened-before the entering event. Similarly, a letter is sent before it is received; the sending event happened-before the receiving event. These everyday actions illustrate how causal ordering can be inferred without a shared clock.

Algorithm / Steps

1. **Same-process ordering** - If event *a* occurs before event *b* on the same process, then the logical clock value of *a* is less than that of *b*.
2. **Message ordering** - If process *p* sends a message *m* to process *q*, then the send event of *m* happened-before the receive event of *m*.
3. **Transitivity** - If event *a* happened-before event *b* and event *b* happened-before event *c*, then event *a* happened-before event *c*.

Key points

- Asynchronous systems lack a shared global clock, so physical time cannot order events reliably.
- Lamport's happened-before relation provides a partial order based on logical timestamps.
- The relation is defined by three rules: same-process ordering, message ordering, and transitivity.
- This logical ordering allows processes to reason about causality without synchronized clocks.

Group Communication

Definition

Group communication refers to the interaction among a set of processes that are connected by reliable communication channels, where a single master or leader coordinates access to shared resources, often using a token mechanism.

Explanation

In this model, each process sends a request to the master to enter a critical section. The master maintains a queue of waiting requests and holds a special token that grants permission to enter the critical section. When a process receives the token, it can enter the critical section; after finishing, it returns the token to the master, which then grants it to the next waiting process. This central solution ensures mutual exclusion among the group.

Example

A token-based mutual exclusion system where a master keeps a queue of requests and a token that allows the holder to access the critical section. The process sends a request, waits for the token, enters the critical section, exits, and sends the token back to the master.

Algorithm / Steps

1. Process sends a request to the master to enter the critical section.
2. Master queues the request and, when the token is available, sends it to the requesting process.
3. Process receives the token and enters the critical section.
4. After completing its work, the process exits the critical section and sends the token back to the master.

Key points

- Processes communicate over reliable channels.
- The master maintains a queue of waiting requests.
- A single token grants exclusive access to the critical section.
- The model assumes no process failures for simplicity.

Causal Order

Take the example that I enter the house only if I unlock. So, it thatmeans,unlockingofaneventthetimeofthat should always be less than thetimewhenyouenterthehouse.So;thatmeans,unlocking happensbeforetheenteringintothehouse. (ReferSlideTime:58:39) The second example is that you receive a letter only after I send it. So, the sendingofa letter happens before the receive of a letter so; that means,. So, this particular way humans are basically using the physical clock and now they are basically ordering the event, for example,hereunlockingfirstandthenenteringthehouseorsendingletterfirst and then receiving it later. So, without using physical clock, if you can obey this causality,thenthatconceptcanbeusedwithouteven. (ReferSlideTime:60:07) The syn

Total Order

Definition

Total order is an ordering of events such that all processes agree on the same sequence, e.g., FIFO order of token granting.

Explanation

The token algorithm ensures that the master distributes the token in FIFO order, guaranteeing that all processes see the same order of critical-section entry.

Example

Token is granted to processes in the order 1, 2, 3, ..., N.

Algorithm / Steps

1. A process requests the token.
2. The master queues the request.
3. The master grants the token to the first process in the queue.
4. The process enters the critical section.
5. The process releases the token.
6. The master returns the token to the next waiting process.

Key points

- Only one token exists, ensuring mutual exclusion.
- The master maintains a FIFO queue of waiting processes.
- Token distribution follows FIFO order, providing a total order of critical-section access.

- Safety, liveness, and fairness are guaranteed.

Global State and Snapshot Recording Algorithms: Introduction

So, if you sum up all the values, it basically will give you that 800 that is the total amount is preserved, hence it is a consistent global state recording algorithm, which is recording the state, which is global state, which is a consistent state. Now, let us see another example. In this example, the algorithm is initiated at this instant of time that is after just after t 0. Now, as far as the marker, the rule one says that it will record its state that is A will be 600, and then it will send the marker. So, when the marker is received by a site S2, it will record its; it will record its state of a channel C 12 as empty. And then, it will apply the marker sending rule, and it will then it will record its state that is B will be equal to

System Model and Definitions

Definition

The asynchronous distributed system model consists of a number of processes, each having a state that includes data, stack, heap, program counter, and registers. The state changes via internal events, send events, and receive events. The system model for recording a global snapshot is a collection of (n) processes (p_1) to (p_n) connected by channels (C_ij). Each channel has a state (SC_ij). Actions performed by a process are modeled as internal events, message send events, and message receive events. The state of a process (p_i) at any instant is denoted by (LS_i).

Explanation

In an asynchronous model, messages are the only means of communication, and delays are unpredictable and unbounded. There is no shared global clock or memory, making clock synchronization non-trivial. Time is crucial for correctness and fairness in distributed and cloud systems; without synchronization, clocks may drift, causing missed events or unfair waiting. The snapshot algorithm must record the global state of a distributed system on-the-fly, without stopping the system, which is challenging because of the lack of a global shared memory, a global common clock, and unpredictable message delays.

Example

In an airline reservation system using the cloud, server x timestamps the purchase of the last ticket at 6 h 25 min, while server y, upon receiving a booking request, records the flight as full at 6 h 20 min. When server z queries both servers, it receives conflicting information because the clocks at x and y are not synchronized.

Key points

- The asynchronous model has no global clock or bounded delays.
- Processes communicate only via message passing; there is no shared memory.
- A snapshot system model includes (n) processes, channels (C_ij), and states (SC_ij).
- Clock synchronization is essential for correctness and fairness in distributed/cloud systems.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

- Global snapshot recording must be performed on-the-fly in the presence of unpredictable message delays.

Snapshot Algorithms for FIFO Channels.

At time t 0, the account of A is 600 dollars, account B is 200 dollars and the channels are empty. At time t 1, site S1 will initiate a transfer of 50 dollars from account A to account B. So, accounting A is decremented by 50 dollars and a request to credit 50 dollars is being sent the message as a message to the account B over a communication channel C 12. So, the account A becomes 550, whereas the account B is not changed. And the request to credit that 50 dollar is in the message, which is in the communication channel C 12, C21 is 0. And so on, we will see this example here in this particular way. So, here as I told you that 50 dollar is being sent, and it will be debited, similarly here in at time t 1. Before time t 2, 80 dollar is bein

Source and licence

This PDF is adapted from **NOC:Cloud Computing and Distributed Systems (NPTEL course book, week 4)** (https://nptel.ac.in/courses/106104182), accessed 2026-09-26.

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).